Getting Started

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Getting Started **2**
Pages . . . 2
What it computes . . . 2
What it is used for . . . 3
How it is run . . . 3
Source and license . . . 3
Code . . . 3
Inputs . . . 3
Routing . . . 4
Management . . . 4
Aquifers . . . 4
Outputs . . . 4
Migration . . . 4
License . . . 5
How to cite . . . 5
Source . . . 5
Versioning . . . 5
Tagging policy . . . 5
Where to find what changed . . . 5
Option 1: Prebuilt executable . . . 6
Option 2: Build from source . . . 6
Supporting tools . . . 6
Verifying the install . . . 7
What it is . . . 7
Prerequisites . . . 7
Run it . . . 7
What you should see . . . 7
If it fails . . . 8
Next . . . 8
The master file: `file.cio` . . . 8
File groups . . . 9
Next . . . 11
What you need . . . 11
Step 1: Delineate the watershed (QSWAT+) . . . 11
Step 2: Create HRUs . . . 11
Step 3: Build the input files (SWAT+ Editor) . . . 12
Step 4: Run . . . 12
Step 5: Look at output . . . 12
Step 6: Calibrate . . . 12
Common first-run problems . . . 12

Getting Started

This section gets you from a fresh machine to a running SWAT+ simulation.

If you have never run the model before, follow the pages in order:

1. Install. Get the SWAT+ executable on your machine.
2. Quick start: Ames_sub1. Run the small reference dataset shipped with the source repository.
3. Project folder anatomy. Understand what is in a SWAT+ project directory and what each group of files does.
4. Your first watershed. Build a project for your own area using QSWAT+ and run it.

If you only want to read about the model first, skip ahead to Model theory.

SWAT+ is a watershed model. It simulates water, sediment, nutrient, pesticide, and crop yield response to climate and management at scales from a single field up to a large river basin.

This section covers what the model is, how it differs from the original SWAT, how to cite it, and how releases are numbered.

Pages

- What is SWAT+
- SWAT vs SWAT+
- Citing and license
- Release notes

The Soil and Water Assessment Tool Plus (SWAT+) is a public-domain watershed model. It is jointly developed by the USDA Agricultural Research Service (USDA-ARS) and Texas A&M AgriLife Research, with contributions from Colorado State University, the USDA-ARS Hydrology and Remote Sensing Laboratory, and other institutions.

SWAT+ simulates the quantity and quality of surface water and groundwater at scales from small watersheds to large river basins. It is used to evaluate the effects of land use, land management practices, soil, climate, and climate change on water resources.

What it computes

For each hydrologic response unit (HRU), routing reach, aquifer, reservoir, and wetland in a watershed, SWAT+ computes a daily mass balance for:

- water (precipitation, evapotranspiration, surface runoff, lateral flow, percolation, return flow, channel routing)
- sediment (detachment, transport, deposition)
- nutrients (nitrogen and phosphorus cycling in soil, plant uptake, transport in surface and subsurface flow, in-stream transformations)
- carbon (soil organic carbon pools)
- pesticides (application, decay, transport)
- salts and other constituents
- plant growth (biomass, yield, water and nutrient stress)

Output is reported daily, monthly, yearly, or as an annual average.

What it is used for

Typical applications:

- assessing the effect of management practices on water yield and water quality
- evaluating soil erosion and non-point source pollution
- estimating crop yield response to climate and management
- supporting watershed planning and total maximum daily load (TMDL) studies
- climate change impact studies on water resources

How it is run

SWAT+ is a Fortran command-line executable. It reads a folder of plain-text input files that describe the watershed, climate, soil, land use, management, and simulation period. It writes output as plain-text files in the same folder.

Most users do not write the input files by hand. They build a project with one of the supporting tools:

- **QSWAT+** is a QGIS plugin that delineates the watershed from a DEM, creates HRUs, and writes the input files.
- **SWAT+ Editor** is a desktop application for inspecting and editing input parameters in an existing project.

After the tools write the project folder, the SWAT+ executable is run from that folder.

Source and license

The source code is hosted at https://github.com/swat-model/swatplus and released under LGPL-2.1.

SWAT+ is the modernised successor to SWAT (Soil and Water Assessment Tool). The science is largely the same. The code structure, input file layout, and connectivity model are different.

The SWAT (2012) column below is sourced from the legacy model repository at `swat-model/swat2012`.

Code

	SWAT (2012)	SWAT+
Language	Fortran (fixed-form)	Fortran free-form (`.f90`)
Structure	Large shared common blocks	Modular, files grouped per process (`aqu_*`, `channel_*`, `hru_*`, `gwflow_*`, …)
Source	Distributed as archives	Git on GitHub, public, LGPL-2.1
Build	Hand-written Makefiles	CMake

The CMake build for SWAT+ uses `-free` (Intel) or `-ffree-line-length-none` (GNU) and glob-compiles `src/*.f90` into a single executable. See `CMakeLists.txt` in the source repository.

Inputs

SWAT2012 used a set of input file extensions tied to subbasins (`.sub`, `.rte`, `.gw`, `.sol`, etc.) with one file per HRU and subbasin.

SWAT+ replaces this with an object-based layout. The watershed is a graph of objects (HRUs, channels, aquifers, reservoirs, point sources, recall objects, decision tables) that are wired together through connectivity files (`.con`). The connectivity slots are defined by the `input_con` type in `src/input_file_module.f90`:

```
hru.con, hru-lte.con, rout_unit.con, gwflow.con, aquifer.con,
aquifer2d.con, channel.con, reservoir.con, recall.con, exco.con,
delratio.con, outlet.con, chandeg.con
```

Inputs are grouped by purpose, not by spatial unit:

- climate: `*.cli`, `*.pcp`, `*.tmp`, `weather-sta.cli`, `weather-wgn.cli`
- land use management: `landuse.lum`, `management.sch`, `cntable.lum`, `cons_practice.lum`, `ovn_table.lum`
- soils: `soils.sol`, `nutrients.sol`
- decision tables: `lum.dtl` (rule-based management triggers)
- hydrology: `hydrology.hyd`, `topography.hyd`
- basin-level: `codes.bsn`, `parameters.bsn`, `time.sim`, `print.prt`
- master index: `file.cio`

Routing

SWAT2012 routed water from subbasin to subbasin in a fixed order based on subbasin numbering.

SWAT+ uses the `*.con` files to describe arbitrary connectivity between HRUs, routing units, channels, aquifers, reservoirs, and exit points. Routing follows the graph defined in those files. This makes it possible to route landscape units (LSUs) within an HRU to a channel via a routing unit, or to send flow to a wetland or reservoir before the channel.

Management

SWAT2012 used `mgt` files with a fixed sequence of operations by date and heat unit.

SWAT+ separates the schedule (`management.sch`) from rule-based triggers (`*.dtl`, decision tables). A decision table evaluates conditions (soil water, plant heat units, day of year, etc.) at each time step and fires an operation when the condition is met. The same schedule can be reused across HRUs with different decision tables driving when operations occur.

Aquifers

SWAT2012 had a single shallow-aquifer storage per subbasin.

SWAT+ supports 1D aquifer objects, defined in `src/aquifer_module.f90` and read by `src/aqu_read.f90`, with one aquifer per landscape unit or group. It also includes a 2D groundwater module rooted in `src/aqu2d_read.f90`, `src/aqu2d_init.f90`, and the `gwflow_*` source files, where the aquifer is discretised in plan view and exchanges water laterally with channels, the unsaturated zone, reservoirs, and other objects.

Outputs

SWAT2012 wrote a fixed set of files (`output.hru`, `output.sub`, `output.rch`, `output.std`).

SWAT+ writes one file per object type and aggregation level, controlled by `print.prt`. Examples from `refdata/Ames_sub1`:

```
basin_wb_aa.txt        basin water balance, annual average
hru_wb_aa.txt          HRU water balance, annual average
basin_cha_aa.txt       channel, annual average
basin_aqu_aa.txt       aquifer, annual average
```

The aggregation suffix is `_day`, `_mon`, `_yr`, or `_aa`. Each object row in `print.prt` toggles `daily`, `monthly`, `yearly`, and `avann` independently with `y` or `n`.

Migration

There is no automatic converter from a SWAT2012 project to a SWAT+ project. The recommended path is to rebuild the project in QSWAT+ from the same DEM, soils, land use, and climate inputs, then transfer calibrated parameter changes by hand.

License

SWAT+ is released under the GNU Lesser General Public License, version 2.1 (LGPL-2.1). The full text is in the `LICENSE` file in the source repository.

How to cite

Cite the software release using the DOI:

> Arnold, J.G., Čerkasova, N., White, M.J., Bailey, R., Thorp, K., Jeong, J., Zhang, X., Ugraskan, T., van Griensven, A., Rathjens, H., Raj, C., Cai, X., Geter, W.F., David, O., Carlson, J.R., Le, K.N. *Soil and Water Assessment Tool Plus (SWAT+)*. https://doi.org/10.5281/zenodo.14983534

The DOI `10.5281/zenodo.14983534` resolves to the current release; each tagged release also has its own DOI on Zenodo.

The machine-readable citation lives in `CITATION.cff` in the source repository. GitHub renders a "Cite this repository" button from this file.

Source

- Code: https://github.com/swat-model/swatplus
- Older versions (pre-GitHub): https://bitbucket.org/blacklandgrasslandmodels/modular_swatplus

SWAT+ releases are tagged on the GitHub releases page. Each tag corresponds to a Zenodo DOI under the concept DOI `10.5281/zenodo.14983534`.

Versioning

Releases are identified by a git tag. Tag names look like `61.0` or `61.1` for the current series, and earlier revisions in the reference data appear as `60.5.7`. The tag is read at configure time by `git describe --tags` (see `CMakeLists.txt`) and projected into the executable name and into the banner string in `main.f90`.

The executable name produced by CMake follows the pattern:

```
swatplus-<tag>-<compiler>-<arch>[-Rel|-Dbg]
```

where `<compiler>` is `gnu`, `ifo` (Intel `ifort`), or `ifx` (Intel LLVM), and `<arch>` is the host architecture. On Windows the executable has a `.exe` extension. This is defined in `CMakeLists.txt` as:

```
set(SWATPLUS_EXE "swatplus-${SWAT_VERSION}-${FFC}-${AR}${TY}")
```

A build between tags resolves to a `git describe` string of the form `<tag>-<n>-g<sha>`, for example `61.1-1-g92b4413`. See `doc/Tagging.md` in the source repository.

Tagging policy

See Tagging and releases in the developer guide for the rules a contributor follows when cutting a new tag.

Where to find what changed

For each release, the GitHub release page lists merged pull requests since the previous tag. The full commit history is available via:

```
git log <previous-tag>..<current-tag> --oneline
```

A change log curated for users (input file changes, output format changes, behaviour changes) is maintained in this section as releases are cut.

You have two options.

1. **Download a prebuilt executable** if you only want to run SWAT+.
2. **Build from source** if you want to modify the model or use a development version.

Option 1: Prebuilt executable

Prebuilt binaries for Windows, Linux, and macOS are attached to each GitHub release.

1. Go to https://github.com/swat-model/swatplus/releases.
2. Pick the latest release (or a specific tagged version).
3. Under **Assets**, download the file that matches your operating system and architecture.

Binaries are named `swatplus-<version>-<compiler>-<os>_<arch>` with an optional `-Rel` or `-Dbg` suffix and a `.exe` extension on Windows. The compiler tag is `gnu` (gfortran), `ifo` (Intel `ifort`), or `ifx` (Intel `ifx`). Examples:

- Linux x86_64, gfortran: `swatplus-3.0.4-gnu-lin_x86_64`
- Windows AMD64, gfortran: `swatplus-3.0.4-gnu-win_amd64.exe`
- macOS arm64, gfortran: `swatplus-3.0.4-gnu-mac_arm64`

Put the file somewhere on your `PATH`, or keep it next to your project folder. On Linux and macOS, make it executable:

```
chmod +x swatplus-3.0.4-gnu-lin_x86_64
```

Run it once with no arguments to confirm it starts:

```
./swatplus-3.0.4-gnu-lin_x86_64
```

It will look for input files in the current directory and exit if it does not find them. That is the expected behavior outside of a project folder.

Option 2: Build from source

You need:

- `git`
- `cmake` (3.22 or newer)
- a Fortran compiler: `gfortran`, Intel `ifort`, or Intel `ifx`
- `make` or `ninja`

Clone the source:

```
git clone https://github.com/swat-model/swatplus.git
cd swatplus
```

Configure and build:

```
cmake -B build
cmake --build build -j 8
```

The executable is written to `build/` using the `swatplus-<version>-<compiler>-<os>_<arch>` pattern (with `.exe` on Windows). To pick a specific compiler at configure time, pass `-D CMAKE_Fortran_COMPILER=gfortran` (or `ifort`, `ifx`).

To install it system-wide:

```
cmake --install build --prefix ~/bin
```

For platform-specific build instructions (Visual Studio on Windows, VS Code Codespaces, Intel compilers), see the Developer guide.

Supporting tools

To build SWAT+ inputs from a DEM, soil map, and land use map, install:

- **QSWAT+** (QGIS plugin): https://swat.tamu.edu/software/plus/. Delineates the watershed, defines HRUs, and writes the input files.
- **SWAT+ Editor** (desktop app): edits parameters in an existing project. Distributed from the same page.

The SWAT+ executable itself does not need either of these to run. They produce the input files that the executable reads.

Verifying the install

Use the reference dataset in `swatplus/refdata/Ames_sub1` to confirm the executable runs end-to-end. See Quick start: Ames_sub1.

Run a complete SWAT+ simulation in a few minutes using the small reference dataset shipped with the source repository.

What it is

`Ames_sub1` is a twelve-HRU dataset near Ames, Iowa. It simulates 1975 through 2020 on a daily time step (see `time.sim`). The dataset is in the SWAT+ source tree at `refdata/Ames_sub1`.

Prerequisites

- A SWAT+ executable. See Install.
- A copy of the `Ames_sub1` folder. If you cloned the source repository, it is at `swatplus/refdata/Ames_sub1`. Otherwise, download it from the link above.

Run it

1. Copy the executable into the `Ames_sub1` folder, or note its full path.
2. From a terminal, change into the `Ames_sub1` folder:

   ```
   cd path/to/Ames_sub1
   ```

3. Run the model:

   ```
   # Linux / macOS
   ./swatplus-3.0.4-gnu-lin_x86_64

   # Windows
   swatplus-3.0.4-gnu-win_amd64.exe
   ```

A simulation of this size takes a few seconds to a minute depending on your machine.

What you should see

While running, SWAT+ writes progress to the console and to `simulation.out`. When it finishes successfully, the last lines of `simulation.out` contain:

```
Execution successfully completed
```

The same line is written to `success.fin`.

The folder now contains a set of output files. With the default `print.prt` for this dataset, every `objects` row is set to `avann = y`, so each enabled object group writes an annual-average file. The annual-average files that appear in `refdata/Ames_sub1` after a successful run include:

```
basin_wb_aa.txt           basin water balance
basin_nb_aa.txt           basin nutrient balance
basin_ls_aa.txt           basin landscape (sediment)
basin_pw_aa.txt           basin plant and water stress
basin_aqu_aa.txt          basin aquifer
basin_cha_aa.txt          basin channel (legacy routing)
basin_sd_cha_aa.txt       basin channel (SWAT-DEG routing)
basin_res_aa.txt          basin reservoir
basin_psc_aa.txt          basin point-source / recall
basin_crop_yld_aa.txt     basin crop yields
crop_yld_aa.txt           crop yields (HRU detail)
hru_wb_aa.txt             HRU water balance
hru_nb_aa.txt             HRU nutrient balance
hru_ls_aa.txt             HRU landscape balance
hru_pw_aa.txt             HRU plant and water stress
```

Open `basin_wb_aa.txt` to see the simulated water balance. The columns are documented in Water balance outputs.

If it fails

- **`Error opening file...`** Check that you are running the executable from inside the `Ames_sub1` folder, not from its parent.
- **`Permission denied`** on Linux or macOS. Run `chmod +x` on the executable.
- The model exits before writing output. Check `diagnostics.out` and `checker.out` for the first error.

For other errors, see FAQ and troubleshooting.

Next

- Read Project folder anatomy to learn what each file in `Ames_sub1` does.
- Walk through the dataset slowly in the Ames_sub1 walkthrough.
- Build your own watershed in Your first watershed.

A SWAT+ project is a single flat folder of plain-text files. The executable reads everything in that folder and writes outputs back into the same folder.

There are no subdirectories. The grouping below is by purpose, not by location.

The master file: `file.cio`

`file.cio` is the index. It tells SWAT+ which file names to read for each category of input. Every line names a category and the file(s) to read. The literal token `null` means "no file for this slot". Full column reference: `file.cio`.

Excerpt from `Ames_sub1/file.cio`:

```
file.cio: AMES
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli           tmp.cli
connect           hru.con           null              null              null              null
hru               hru-data.hru      null
aquifer           nbull             null
hydrology         hydrology.hyd     topography.hyd    null
hru_parm_db       plants.plt        fertilizer.frt    tillage.til       pesticide.pes     null
ops               harv.ops          graze.ops         irr.ops           chem_app.ops      fire.ops
lum               landuse.lum       management.sch    cntable.lum       cons_practice.lum ovn_table.lum
init              plant.ini         soil_plant.ini    null              null              null
```

```
soils             soils.sol         nutrients.sol     null              null
decision_table    lum.dtl           null              null              null
regions           ls_unit.ele       null              null              null              null
```

Read this file first when you open a project. It tells you what every other file is for.

File groups

Simulation control

File	What it controls
`time.sim`	Start and end dates, time step
`print.prt`	Which output files to write, and at which aggregation (`_day`, `_mon`, `_yr`, `_aa`)
`object.cnt`	Object counts (how many HRUs, channels, aquifers, etc.)

Basin parameters

File	What it controls
`codes.bsn`	Method switches (which equation set to use for ET, runoff, routing, …)
`parameters.bsn`	Basin-wide numeric parameters

Climate

File	What it controls
`weather-sta.cli`	List of weather stations and which files belong to each
`weather-wgn.cli`	Weather generator parameters (used to fill missing data)
`pcp.cli`, `tmp.cli`, …	Indexes pointing to the actual `.pcp`, `.tmp`, etc. data files
`<name>.pcp`, `<name>.tmp`	The daily precipitation and temperature time series

Spatial objects and connectivity

File	What it controls
`hru.con`	HRU connectivity (which object each HRU drains to)
`chandeg.con`, `aquifer.con`, `reservoir.con`	Channel, aquifer, reservoir connectivity
`rout_unit.con`	Routing unit connectivity
`hru-data.hru`	Per-HRU pointers to topography, soils, land use, management

Hydrology

File	What it controls
`hydrology.hyd`	Per-HRU hydrologic parameters
`topography.hyd`	Per-HRU slope, slope length, field length

Soils

File	What it controls
`soils.sol`	Soil layer properties (bulk density, available water, conductivity, …)
`nutrients.sol`	Initial soil nutrient pools

Land use and management

File	What it controls
`landuse.lum`	List of land use scenarios and the files that define each
`management.sch`	Operation schedules (planting, harvest, fertilizer, …)
`cntable.lum`	Curve number lookup
`cons_practice.lum`	Conservation practice factor
`ovn_table.lum`	Manning's n for overland flow
`lum.dtl`	Decision tables that trigger operations based on conditions

Plant, fertiliser, tillage, pesticide databases

File	What it controls
`plants.plt`	Plant parameter database
`fertilizer.frt`	Fertiliser composition
`tillage.til`	Tillage operation effects
`pesticide.pes`	Pesticide properties

Initial conditions

File	What it controls
`plant.ini`	Starting plant communities by HRU
`soil_plant.ini`	Starting soil water, residue, plant biomass

Aquifers, reservoirs, channels

File	What it controls
`aquifer.aqu`	Aquifer parameters
`reservoir.res`, `initial.res`	Reservoir parameters and initial state
`channel-lte.cha`, `hyd-sed-lte.cha`	Legacy channel routing
`hydrology.cha`, `sediment.cha`, `nutrients.cha`	SWAT+ channel routing parameters

Outputs (written by the model)

Files ending in `_day.txt`, `_mon.txt`, `_yr.txt`, or `_aa.txt`. The prefix tells you the spatial scope (`basin_`, `region_`, `lsunit_`, `hru_`, `channel_`, `aquifer_`, `reservoir_`, …) and the underscore code tells you the variable group:

Code	Variables
`_wb`	water balance
`_nb`	nutrient balance
`_ls`	landscape (sediment)
`_pw`	plant and water stress
`_aqu`	aquifer
`_cha` / `_sd_cha`	channel
`_res`	reservoir

See Output reference for column details.

Diagnostics

File	What it tells you
`simulation.out`	Run progress and successful completion message
`diagnostics.out`	Warnings and unexpected values during the run
`checker.out`	Validation messages produced before the simulation starts
`area_calc.out`, `files_out.out`, `erosion.txt`	Setup summaries

Always check these three (`simulation.out`, `diagnostics.out`, `checker.out`) when a run does not behave as expected.

Next

The full per-file reference lives in Input reference, with one page per file and a real example pulled from `Ames_sub1`.

This walks through the workflow to build a SWAT+ project for an area of your own, run it, and look at the output. It does not click through QSWAT+ screen by screen. It covers the steps and what each one produces so you know what to expect.

What you need

Data:

- A digital elevation model (DEM) covering your area, in a projected coordinate system, with no holes.
- A soil map, either SSURGO (US), HWSD (global), or a custom raster paired with a soil property lookup table.
- A land use map, either NLCD (US), ESA WorldCover (global), or a custom raster paired with a SWAT+ land use lookup table.
- A daily climate record (precipitation and temperature at minimum, optionally solar radiation, wind, humidity) from one or more stations covering the simulation period.
- An outlet point (the gauge or stream location you want to drain to).

Software:

- QGIS with the QSWAT+ plugin. See Install.
- SWAT+ Editor.
- The SWAT+ executable.

Step 1: Delineate the watershed (QSWAT+)

In QGIS, create a new QSWAT+ project and load the DEM. QSWAT+ will:

1. Fill sinks in the DEM.
2. Compute flow direction and accumulation.
3. Build the stream network at a threshold drainage area you choose.
4. Subdivide the watershed into subbasins at stream junctions.
5. Snap your outlet point to the stream network.

Output: a polygon layer of subbasins and a polyline layer of channels (reaches).

Step 2: Create HRUs

Overlay the land use map, soil map, and a slope class layer on the subbasins. Each unique combination of (subbasin, land use, soil, slope class) is a candidate HRU. QSWAT+ writes the HRUs and the connectivity files.

You can keep all candidate HRUs (high spatial detail, slow run) or filter by area threshold (faster, less detail).

Output: HRU definitions and `*.con` connectivity files.

Step 3: Build the input files (SWAT+ Editor)

Open the project in SWAT+ Editor. The editor will:

1. Import climate station data and write the `.cli`, `.pcp`, `.tmp`, and `weather-wgn.cli` files. Stations without data get filled by the weather generator.
2. Populate soil layer parameters from the soil property table.
3. Populate plant, fertiliser, tillage, and pesticide databases.
4. Let you assign management schedules to each land use.
5. Write the final input folder.

Set the simulation period in `time.sim` (typically include a one- to three-year warm-up at the start) and the output toggles in `print.prt`.

Output: a folder of plain-text SWAT+ inputs, ready to run.

Step 4: Run

Copy the SWAT+ executable into the project folder. From a terminal:

```
cd path/to/your/project
./swatplus-<version>-gnu-lin_x86_64
```

Watch `simulation.out` for the `Execution successfully completed` message. Check `diagnostics.out` and `checker.out` for warnings.

Step 5: Look at output

Start with the basin water balance:

`basin_wb_aa.txt`

The annual-average precipitation, evapotranspiration, surface runoff, lateral flow, percolation, and water yield should be in the range you expect for your climate. If precipitation is wildly off, your climate file mapping is wrong. If ET dominates everything, check land use and plant parameters.

Then look at the channel output (`basin_sd_cha_aa.txt` or the per-channel `channel_sd_aa.txt`) for streamflow.

If you have measured streamflow at the outlet, compare to `channel_sd_day.txt` for the gauge channel.

Step 6: Calibrate

The first run is rarely a good match to observed data. Move on to Calibration to adjust parameters and improve the fit.

Common first-run problems

- **No precipitation anywhere.** The climate station coordinates are outside the watershed extent, so QSWAT+ assigned every HRU to the weather generator instead of the station. Check `weather-sta.cli`.
- **Crops never grow.** The management schedule is not assigned, or the planting date is outside the simulation period.
- **All flow is surface runoff.** Curve numbers are too high. Check `cntable.lum` and the soil hydrologic group.
- **Streamflow is zero downstream.** A channel object has no upstream connection in `chandeg.con` or `hru.con`. Inspect the connectivity.